# Terraform Provisioners

## 1. Introduction

Terraform Provisioners allow you to execute scripts or commands on a resource after it is created. They act as a bridge between Terraform's infrastructure provisioning and manual configuration tasks. Provisioners are useful for installing software, configuring instances, copying files, or executing custom scripts. They are especially helpful when configuration management tools (like Ansible, Chef, or Puppet) are not available, or when quick setup tasks are required.

## 2. What is a Terraform Provisioner & its uses?

- Terraform **Provisioners** help **execute scripts or commands** on a resource after it is created.
- Helps in copying a file or directory from local to the remote resource
- Used for tasks like **installing software, configuring instances, or running custom scripts**.
- Acts as a bridge between **Terraform and manual configuration**.

## 3. Uses of Terraform Provisioners

Terraform Provisioners can be used to:

- Execute scripts or commands after resource creation.
- Copy files or directories from the local machine to a remote resource.
- Install required software packages on instances.
- Perform quick, one-time configurations.

## 4. Types of Provisioners in Terraform

| Provisioner Type | Purpose |
|---|---|
| local-exec | Runs a command on the local machine where Terraform is installed |
| remote-exec | Runs a command on the remote machine (e.g., EC2 instance, VM) |
| File | Transfers files from the local machine to a remote machine |

## 5. Types of Provisioners in Terraform

### 5.1 Local-Exec Provisioner

The local-exec provisioner executes commands on the local machine where Terraform is running.

It is commonly used for logging, triggering local scripts, or sending notifications.

Example: Save instance details locally

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "example" {
  ami           = "ami-08b5b3a93ed654d19"
  instance_type = "t2.micro"
  key_name      = "my-new-terraform"
  subnet_id     = "subnet-04bfefe4d0a6d3cac"
  vpc_security_group_ids = ["sg-0c7796a163958a667"]

  provisioner "local-exec" {
    command = "echo Instance ${self.id} has been created >
instance_id.txt"
  }

  tags = {
    Name = "LocalExecExample"
  }
}
```

---

Explanation:

- Executes a local command after instance creation.
- Saves the instance ID to a local file named 'instance_id.txt'.
- Runs only on the local machine where Terraform is executed.

### 5.2 Remote-Exec Provisioner

The remote-exec provisioner runs commands directly on a remote resource (e.g., EC2 instance) via SSH or WinRM.

It is ideal for post-deployment software installation and configuration.

Example: Install Apache HTTP Server on EC2

```
provider "aws" {
 region = "us-east-1"
}

resource "aws_instance" "example" {
 ami           = "ami-08b5b3a93ed654d19"
 instance_type = "t2.micro"
 key_name      = "my-new-terraform"
 subnet_id     = "subnet-04bfefe4d0a6d3cac"
 vpc_security_group_ids = ["sg-0c7796a163958a667"]
 associate_public_ip_address = true

 connection {
  type        = "ssh"
  user        = "ec2-user"
  private_key = file("${path.module}/my-new-terraform.pem")
  host        = self.public_ip
 }

 provisioner "remote-exec" {
  inline = [
   "sudo yum update -y",
   "sudo yum install -y httpd",
   "sudo systemctl start httpd",
   "sudo systemctl enable httpd"
  ]
 }

 tags = {
  Name = "RemoteExecExample"
 }
}
```

---

Explanation:

- Connects to the EC2 instance via SSH using a private key.
- Installs and starts the Apache HTTP Server.
- Configures Apache to start automatically on boot.

### 5.3 File Provisioner

The file provisioner is used to transfer files or directories from the local machine to a remote resource.

It is useful for uploading scripts, configuration files, or application files.

Example: Upload a script to EC2 instance

```
provider "aws" {
 region = "us-east-1"
}

resource "aws_instance" "example" {
 ami           = "ami-08b5b3a93ed654d19"
 instance_type = "t2.micro"
 key_name      = "my-new-terraform"
 subnet_id     = "subnet-04bfefe4d0a6d3cac"
 vpc_security_group_ids = ["sg-0c7796a163958a667"]
 associate_public_ip_address = true

 connection {
  type        = "ssh"
  user        = "ec2-user"
  private_key = file("${path.module}/my-new-terraform.pem")
  host        = self.public_ip
 }

 provisioner "file" {
  source      = "local_script.sh"
  destination = "/home/ec2-user/remote_script.sh"
 }

 tags = {
  Name = "FileProvisionerExample"
 }
}
```

---

Explanation:

- Transfers a file from the local machine to the EC2 instance.
- Source file: local_script.sh
- Destination file: /home/ec2-user/remote_script.sh

## 6. Provisioner Behaviour & Lifecycle

- Provisioners run after the resource is created by default.
- If a provisioner fails, Terraform marks the resource as 'tainted' (requiring recreation).

## 7. When to Use Provisioners

- When configuration automation tools are not available.
- When you need to execute custom commands after infrastructure creation.
- For quick, one-time setup tasks on resources.

## 8. When to Avoid Provisioners

- When user data (AWS advance section) scripts can handle the task.
- When configuration management tools (like Ansible) are available.

## 9. What is User Data in AWS?

- User Data in AWS allows you to specify a script that runs automatically when an EC2 instance is launched.
- It is commonly used for automating setup tasks like installing software, configuring settings, or running commands.
- User Data scripts run only once during the first boot of the instance.